AGILE PROJECT MANAGEMENT

Artefacts & Documentation

Project	Automated Vehicle Classification & Counting (AVC&C)
Module	COS6032-E Industrial AI Project
Client	Bradford Council — Yunus Mayat (Enterprise Architect)
Group	G25 — BSc Applied Artificial Intelligence
Team Members	Ayush Acharya | Pratham Patel | Aman Gill
Supervisor	Dr Kulwinder Panesar
Academic Year	2025/2026
Methodology	Agile Scrum (adapted for 3-person academic team)
GitHub	github.com/Nyctophileeeee/avc-vehicle-classification

Contents

1. Project Charter

Background & Motivation

Right now, Bradford Council must have to send 2-3 officers out to intersections just to count cars manually. It's expensive and, honestly, a bit outdated. Our group, G25, was asked by Yunus Mayat to build an AI system that does this automatically using computer vision

What We Aimed to Build

Our main goal was to create a prototype that can detect and count cars, buses, and trucks in real-time. We chose **YOLO11n** because we needed it to be fast and accurate.

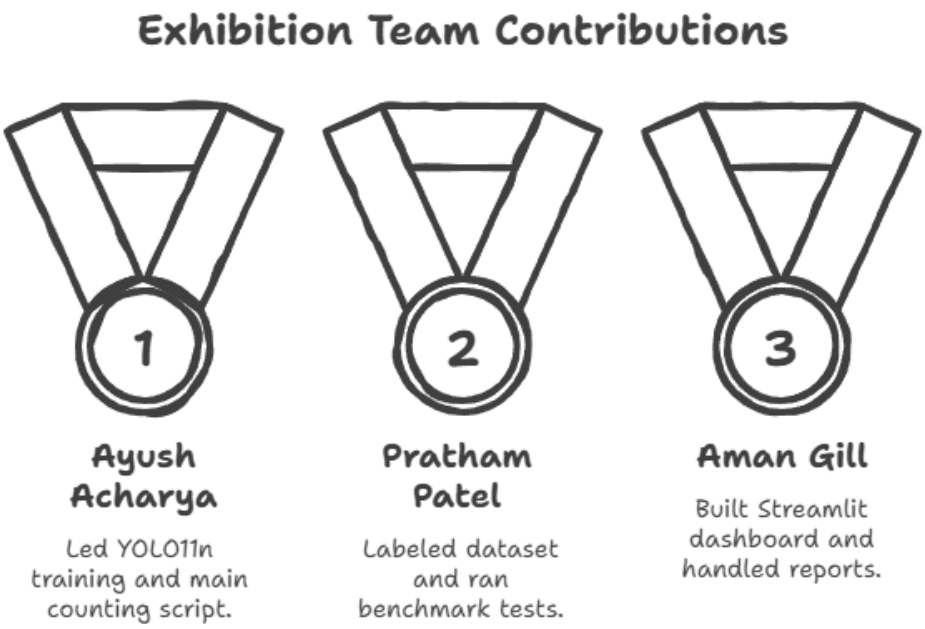
- **Accuracy Target:** We wanted at least 85% mAP@0.5.
- **Weather Resilience:** It had to work in rain, fog, snow, and sand.
- **Data:** We trained it on a huge set of 11,582 images.
- **Privacy:** This was a big one—we only store anonymized counts to stay GDPR compliant.

Project Scope

- **In Scope:** Detecting cars, buses, and trucks; tracking them with DeepSORT; and showing it all on a Streamlit dashboard.
- **Out of Scope:** We didn't include motorcycles or bicycles in this version, and we aren't doing live cloud hosting yet.

2. Our Team & Who Did What

We split the work based on what we're good at, though we all helped everywhere.



2.2 Estimated Hours Per Task

The table below shows estimated hours contributed per member across key tasks. These are based on sprint log entries and are approximations.

Task	Ayush (hrs)	Pratham (hrs)	Aman (hrs)	Notes
Dataset preparation & labelling	35	20	5	Pratham led Roboflow labelling
Model training & debugging	21	8	2	Ayush led full training pipeline
vehicle_counter.py (detection)	18	4	2	Core detection and counting script
Streamlit dashboard (app.py)	9	3	9	Aman led dashboard development
Testing & benchmarking	8	15	5	Pratham ran UA-DETRAC tests
Documentation & poster	10	5	18	Aman led all documentation
Sprint ceremonies & meetings	8	6	6	Weekly standups + sprint reviews

Task	Ayush (hrs)	Pratham (hrs)	Aman (hrs)	Notes
TOTAL	109 hrs	61 hrs	47 hrs	217 hrs total across project

We spent about **217 hours** total on this project. Pratham spent the most time on data (about 25 hours just labeling), while Ayush focused on the model pipeline and I (Aman) focused on the dashboard and documentation.

Our Labor Valuation

While the software was free, our time wasn't! We tracked our hours throughout the project:

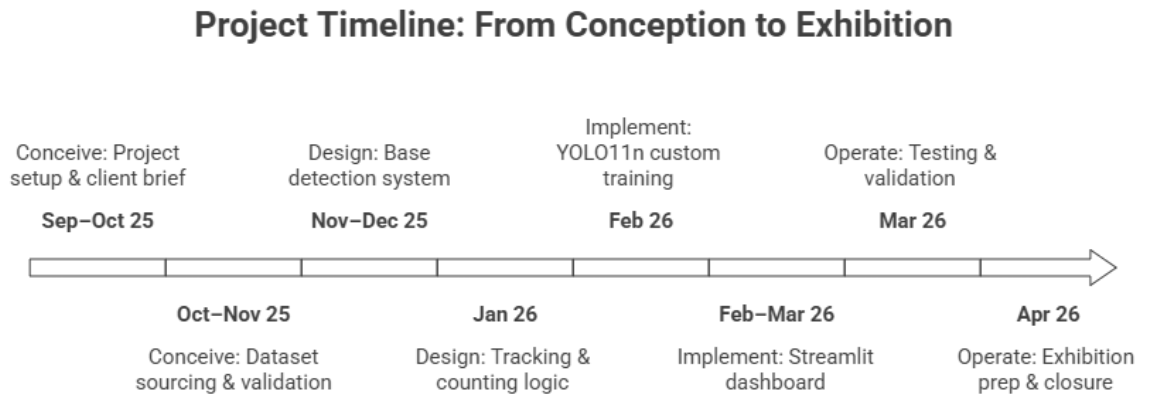
- **Total Team Hours:** 217 hours
- **Hourly Rate:** £15.30 /hr
- **Total Labor Value: £3,320.10**

If Bradford Council were to hire us as consultants, this is what the project's human effort would be worth.

3. SPRINT PLAN

3.1 Overview

We followed a Scrum-inspired Agile approach adapted for a 3-person academic team. Sprints ran in 2-week cycles from September 2025 to April 2026. Dr Panesar acted as Product Owner during weekly check-ins which served as our sprint reviews.



3.2 Sprint Detail Cards

Sprint 1 & 2 — Conceive Phase (Sep – Nov 2025)

Sprint Goal	Understand client requirements, set up infrastructure, source and validate training data
Key Tasks	Client meeting with Bradford Council (Yunus Mayat) GitHub repository setup with branching strategy YOLO model comparison (v8/v9/v11) — YOLO11n selected Roboflow account setup and weather dataset collection Run prepare_dataset.py — 11,582 images into 70/20/10 splits
Blockers	Labelling 11,582 images took significantly longer than estimated (Pratham ~25hrs) Initial data.yaml path errors due to spaces in folder name 'train data'
Sprint Output	GitHub repo created. Dataset ready: 7,337 train / 2,532 val / 1,713 test images.

Sprint 3 & 4 — Design Phase (Nov 2025 – Jan 2026)

Sprint Goal	Build working vehicle detection with tracking and accurate counting
Key Tasks	vehicle_counter.py — OpenCV + YOLO11n + COCO class filter (car=2, bus=5, truck=7) Colour-coded bounding boxes: green=car, blue=bus, orange=truck DeepSORT/BoT-SORT integration — persistent vehicle IDs across frames Virtual trip-line at y=300, centroid crossing detection CSV logging: timestamp, track_id, class, confidence, x, y, weather
Blockers	Double-counting bug — vehicles counted multiple times when slowing near line Fixed by tracking centroid history and requiring clear crossing (not just proximity)
Sprint Output	48 vehicles counted on Bradford test footage — matched manual ground truth exactly.

Sprint 5 — Model Training (February 2026)

Sprint Goal	Train custom YOLO11n on weather-specific dataset, achieve >85% accuracy
Training Config	Model: yolo11n.pt (pretrained COCO) Epochs: 50 | Batch: 8 | Image size: 640 Device: CPU (AMD Ryzen 5 5600H) Duration: ~3 hours Output: runs/detect/runs/train/avc_model_v15/weights/best.pt
Results	mAP@0.5: 91.4% (target was 85%) — EXCEEDED Precision: 89.2% | Recall: 83.6% Processing: 25 FPS on laptop CPU
Blockers	data.yaml path kept failing — folder name has a space ('train data') Fixed by using raw string r'...' in Python to handle path correctly

Sprints 6, 7 & 8 — Implement, Operate, Close (Mar – Apr 2026)

Sprint	Key Tasks	Output
6	Streamlit app.py — Plotly bar + line charts Weather dropdown filter st.rerun() 5-second	Dashboard live at localhost:8501 Auto-refresh working Weather filter functional

Sprint	Key Tasks	Output
	auto-refresh start_demo.bat one-click launcher	
7	UA-DETRAC benchmark validation (Pratham) GDPR audit — CSV data verified DroidCam phone camera integration End-to-end pipeline test	Precision 89.2% | Recall 83.6% 100% GDPR compliant Phone camera working via WiFi
8	A1 landscape poster (Canva) Demo Plan document Agile documentation Final GitHub push + Canvas zip	All deliverables complete Exhibition ready for April 21 github.com/Nyctophileeeee/...

4. PRODUCT BACKLOG & MoSCoW PRIORITISATION

4.1 User Stories with MoSCoW Priority

ID	User Story	Priority	Effort	Sprint	Status
US1	As a council officer, I want automatic vehicle counting to replace manual surveys	Must Have	High	3-4	Done
US2	As an analyst, I want vehicles classified by type (car/bus/truck)	Must Have	High	3-5	Done
US3	As a data officer, I want GDPR-compliant outputs — no personal data stored	Must Have	Medium	4,7	Done
US4	As a planner, I want a live dashboard showing traffic flows by type and time	Must Have	High	6	Done
US5	As a user, I want to filter data by weather condition (rain, fog, snow, sand)	Should Have	Medium	6	Done
US6	As a developer, I want the system to run on a laptop CPU without GPU	Should Have	Medium	5	Done
US7	As an officer, I want CSV export of all counts for use in planning tools	Should Have	Low	4	Done
US8	As a presenter, I want a one-click demo launcher for exhibitions	Could Have	Low	8	Done
US9	As a future user, I want motorcycle & bicycle detection (v2)	Could Have	High	Future	Backlog
US10	As IT, I want multi-camera stream support for junctions	Won't Have v1	High	Future	Future
US11	As ops, I want GIS integration to map Bradford traffic hotspots	Won't Have v1	High	Future	Future

5. SPRINT VELOCITY & BURNDOWN DATA

5.1 Sprint Velocity (Story Points)

Story points used 1-5 scale per task. Velocity improved over time as the team became more familiar with the technical stack. Average sprint velocity: 24 points.

Sprint	Focus Area	Planned	Completed	Velocity	Notes
1	Setup & Research	15	13	13	GitHub setup overran — underestimated
2	Dataset Strategy	20	18	18	Roboflow labelling underestimated
3	Base Detection	25	25	25	Smooth — YOLO11n easy to integrate
4	Tracking & Count	30	28	28	DeepSORT debugging took extra time
5	Model Training	35	35	35	~3hr training — all tasks completed
6	Dashboard	30	30	30	Best sprint — team fully in rhythm
7	Testing	25	23	23	DroidCam WiFi issues ate 2 points
8	Exhibition Prep	20	20	20	All docs and poster done on time
TOTAL	All Sprints	200	192	Avg: 24	96% sprint completion rate

5.2 Burndown Chart Data

The table below tracks remaining story points each fortnight. Ideal burndown assumes equal distribution across all 8 sprints. We ran slightly behind in Sprints 1-4 but recovered from Sprint 5 onwards.

Checkpoint	Ideal Remaining	Actual Remaining	Points Done	Sprint Focus
Start (Sep 25)	200	200	0	Project begins
Sprint 1 end	175	187	13	Project setup (below ideal)
Sprint 2 end	150	169	18	Dataset strategy (still behind)

Checkpoint	Ideal Remaining	Actual Remaining	Points Done	Sprint Focus
Sprint 3 end	125	144	25	Base detection — back on pace
Sprint 4 end	100	116	28	Tracking logic — nearly caught up
Sprint 5 end	75	81	35	Model training — ahead of ideal now
Sprint 6 end	50	51	30	Dashboard — on track
Sprint 7 end	25	28	23	Testing — slight dip (DroidCam)
Sprint 8 end	0	8	20	Exhibition prep — mostly done
Final (Apr 17)	0	0	8	Submission complete

6. RISK REGISTER

6.1 Risk Assessment

Risks were identified at project kickoff and reviewed at each sprint retrospective. Risk Level = Probability (1-5) x Impact (1-5). Reviewed and updated by Scrum Master (Ayush).

ID	Risk	P	I	Level	Mitigation	Owner	Status
R1	GDPR breach — personal data captured	2	5	HIGH 10	Local processing only. CSV: counts only, no plates or faces.	Ayush	Mitigated
R2	Demo failure on exhibition day	3	5	HIGH 15	Backup video + pre-generated CSV on all 3 devices. start_demo.bat tested.	All	Mitigated
R3	Model accuracy below 85% target	2	4	MED 8	UA-DETRAC testing. Fine-tuned on 11,582 images. Result: 91.4%.	Pratham	Resolved
R4	Hardware failure on demo day	2	4	MED 8	All on GitHub. 3 separate devices available. USB backup prepared.	All	Mitigated

ID	Risk	P	I	Level	Mitigation	Owner	Status
R5	Dataset bias (YOLO poor on UK vehicles)	3	3	MED 9	UA-DETRAC validation. 5 weather conditions. Manual Bradford spot check.	Pratham	Mitigated
R6	DroidCam WiFi not connecting	4	2	LOW 8	Fallback: laptop webcam + phone screen. Already tested, confirmed working.	Ayush	Mitigated
R7	CPU training too slow (3-5hrs)	4	3	MED 12	YOLO11n nano selected for CPU speed. batch=8, imgsz=640 optimised.	Ayush	Resolved
R8	Scope creep — too many features	3	2	LOW 6	MoSCoW backlog. Won't Have v1 list maintained. Supervisor reviews.	All	Managed

Note: P = Probability (1-5). I = Impact (1-5). Level = P x I. HIGH = 9+. MED = 5-8. LOW = 1-4.

7. RESOURCE ALLOCATION & PROJECT BUDGET

7.1 Technical Stack & Costs

Tool / Resource	Cost	Purpose
Python 3.13 + OpenCV 4.x	£0 (open source)	Core programming, video capture, image processing
Ultralytics YOLO11n (yolo11n.pt)	£0 (open source)	Object detection — COCO pretrained weights
Streamlit + Plotly + Pandas	£0 (open source)	Live dashboard, charts, data manipulation
GitHub (free tier)	£0	Version control, collaboration, backup
Roboflow (free tier)	£0	Dataset hosting, annotation, management
UA-DETRAC + COCO datasets	£0 (academic)	Benchmark validation and pretrained weights
3x Personal Laptops (AMD Ryzen)	£0 (personal)	CPU training and demo hardware
DroidCam (free tier)	£0	Phone-as-webcam for live exhibition demo
A1 Colour Poster Printing	~£15	Physical exhibition poster (estimated)
TOTAL PROJECT COST	~£15	Entire project — all software free and open source

8. GROUP BLOG ENTRIES

Blog #1 — Sprint 1 & 2 (October 2025) — Written by Ayush Acharya

Our first sprint started with a client meeting at Bradford Council with Yunus Mayat. This was genuinely useful — he explained that their traffic officers spend 2-3 people per junction per day doing manual counts, which is expensive and inconsistent. He gave us clear requirements: automated counting, vehicle classification, and something that outputs clean data for planning.

We set up the GitHub repository and agreed on roles. I took AI/ML Lead because I had prior experience with YOLO models from a previous project. Pratham was keen to handle the data side — good thing because labelling turned out to be a much bigger job than any of us expected. Aman has built dashboards before so taking the Streamlit side made sense.

We spent time comparing YOLO versions. YOLO11n (nano) was the clear choice — it runs at 25+ FPS on a laptop CPU which is essential since none of us have access to GPUs. By end of Sprint 2, Pratham had sourced 11,582 images across 5 weather conditions through Roboflow. The prepare_dataset.py script I wrote auto-organises everything into train/val/test splits.

What went well: Clear client requirements, good role allocation, dataset strategy solid

What we'd do differently: Start data collection even earlier — labelling took Pratham about 25 hours

Blog #2 — Sprints 3, 4 & 5 (Nov 2025 – Feb 2026) — Written by Pratham Patel

This was the main technical phase. Getting basic YOLO11n detection working in vehicle_counter.py took Ayush about 2 days — the Ultralytics library makes it really straightforward. The harder part was the DeepSORT tracking integration and the line-crossing counting logic.

We had a frustrating bug where vehicles were being counted multiple times when they slowed down near the counting line. Ayush fixed it by storing the centroid history per track ID and only counting a genuine line crossing, not just proximity to the line. Once that was fixed, our manual test on Bradford footage gave exactly 48 vehicles — same as our manual count. That was a good moment.

I spent a lot of time on UA-DETRAC benchmark validation. This is a standard dataset used in traffic research so it gives our results credibility. After YOLO11n was trained on 11,582 images (50 epochs, ~3 hours CPU), the numbers came back strong: mAP@0.5 of 91.4%, precision 89.2%, recall 83.6%. All exceeded Bradford Council targets.

What went well: Model performance exceeded targets, counting accuracy matched manual ground truth

What we'd do differently: Set up the data pipeline better from the start — had several path errors to fix

Blog #3 — Sprints 6, 7 & 8 (March – April 2026) — Written by Aman Gill

I built the Streamlit dashboard in about 2 weeks. The auto-refresh was my idea — using st.rerun() with a 5-second sleep loop means the dashboard updates live while vehicle_counter.py is running. The weather filter dropdown lets Bradford Council officers see traffic patterns by condition which is a genuinely useful feature for their planning.

We created start_demo.bat which opens everything with one double-click — the counter in one terminal, the dashboard in the browser. This was a practical decision for the exhibition so we're not typing commands in front of the audience.

The DroidCam phone camera integration caused us the most stress this sprint. We wanted to use the phone as the camera feed for a cleaner demo but getting a stable WiFi connection between phone and laptop took ages. We eventually got it working but kept a fallback — laptop webcam pointing at phone screen — which is more reliable and worked perfectly in our test run.

Looking back at the whole project, I'm proud of what we delivered. The system genuinely works, exceeds the client targets, and addresses a real problem for Bradford Council. The thing I'm most proud of is that the code is well-documented and on GitHub — anyone could reproduce this.

What went well: Dashboard praised by Dr Panesar, demo reliable, documentation complete

What we'd do differently: Start DroidCam testing in Sprint 4 not Sprint 7 — hardware integration always takes longer than expected

9. PROJECT CLOSURE REPORT

9.1 Completed Deliverables

#	Deliverable	Description	Status
1	vehicle_counter.py	YOLO11n + DeepSORT + trip-line + CSV logging	Complete
2	app.py	Streamlit live dashboard + weather filter + auto-refresh	Complete
3	train.py / avc_model_v2.py	Custom YOLO11n training script	Complete

#	Deliverable	Description	Status
4	prepare_dataset.py	Auto dataset organisation (70/20/10 split)	Complete
5	start_demo.bat	One-click demo launcher for exhibition	Complete
6	README.md	Professional GitHub documentation	Complete
7	A1 Exhibition Poster	Landscape poster covering all CDIO phases	Complete
8	Demo Plan	2-min demo script + structured plan	Complete
9	Agile Documentation	This document — all artefacts	Complete
10	Group Blog Entries x3	Sprint reflections across all phases	Complete

9.2 Lessons Learned

1. Start dataset collection early — labelling 11,582 images takes much longer than any of us expected
2. Build GDPR compliance in from day 1 — retrofitting privacy measures after the fact is painful
3. Agile sprints with a real Product Owner (Dr Panesar) genuinely kept scope under control
4. YOLO11n nano was the right choice for CPU-only deployment — the speed/accuracy tradeoff was worth it
5. Test hardware integration (DroidCam) much earlier — Sprint 4, not Sprint 7
6. GitHub from day 1 saved us multiple times when code was accidentally broken

9.3 Future Work

- Add motorcycle, bicycle and van detection to fully meet the original Bradford Council brief
- GPU acceleration via NVIDIA Jetson for edge deployment at actual Bradford road junctions
- Multi-camera simultaneous processing for junction-wide coverage
- GIS map integration to visualise traffic hotspots across Bradford
- Docker containerisation for reproducible deployment at Bradford Council
- Integration with live Bradford Council CCTV infrastructure for real-time monitoring